Código fuente · Parcial 2 · Programación III

Universidad del Quindío · Problemas 1 y 2 en Elixir · 26 archivos (.ex y .exs) · 2424 líneas

- **Configuración y scripts de ejecución**
  - mix.exs (21 líneas)
  - .formatter.exs (1 línea)
  - problema1.exs (11 líneas)
  - problema2.exs (11 líneas)
  - ejecutar_todo.exs (20 líneas)
  - menu.exs (54 líneas)
- **Módulos del curso (Util y Util2)**
  - lib/util/util.ex (107 líneas)
  - lib/util/util2.ex (339 líneas)
- **Módulos comunes**
  - lib/comun/entrada.ex (84 líneas)
  - lib/comun/salida.ex (73 líneas)
  - lib/comun/lista.ex (37 líneas)
- **Problema 1: rutas de entrega**
  - lib/problema1/tramo.ex (103 líneas)
  - lib/problema1/itinerario.ex (49 líneas)
  - lib/problema1/depuracion.ex (62 líneas)
  - lib/problema1/busqueda.ex (106 líneas)
  - lib/problema1/alcance.ex (94 líneas)
  - lib/problema1/problema1.ex (262 líneas)
- **Problema 2: regiones y SVG**
  - lib/problema2/color.ex (49 líneas)
  - lib/problema2/region.ex (61 líneas)
  - lib/problema2/validacion.ex (159 líneas)
  - lib/problema2/regiones.ex (95 líneas)
  - lib/problema2/svg.ex (95 líneas)
  - lib/problema2/problema2.ex (156 líneas)
- **Pruebas (ExUnit)**
  - test/test_helper.exs (31 líneas)
  - test/problema1_test.exs (181 líneas)
  - test/problema2_test.exs (163 líneas)

Configuración y scripts de ejecución

mix.exs — 21 líneas

```elixir
defmodule Parcial2.MixProject do
  use Mix.Project

  # Proyecto del Parcial 2 de Programación III (Universidad del Quindío).
  # Problemas resueltos: 1 (rutas de entrega) y 2 (regiones de un mapa + SVG).
  # No usa dependencias externas: solo Elixir estándar.
  def project do
    [
      app: :parcial2,
      version: "1.0.0",
      elixir: "~> 1.15",
      start_permanent: false,
      deps: []
    ]
  end

  def application do
    # :xmerl (incluido en Erlang/OTP) se usa en las pruebas para verificar que el SVG es XML válido.
    [extra_applications: [:logger, :xmerl]]
  end
end
```

.formatter.exs — 1 línea

```elixir
[inputs: ["{mix,.formatter}.exs", "*.exs", "{lib,test}/**/*.{ex,exs}"]]
```

problema1.exs — 11 líneas

```elixir
# Ejecuta el problema 1 (rutas de entrega).
#
#   mix run problema1.exs                          -> caso datos/problema1/ejemplo
#   mix run problema1.exs datos/problema1/invalido  -> otro caso
#   mix run problema1.exs <entrada> <salida>        -> carpeta de salida propia

case System.argv() do
  [] -> Problema1.main()
  [entrada] -> Problema1.main(entrada)
  [entrada, salida | _] -> Problema1.main(entrada, salida)
end
```

problema2.exs — 11 líneas

```elixir
# Ejecuta el problema 2 (regiones de un mapa + SVG).
#
#   mix run problema2.exs                          -> caso datos/problema2/ejemplo
#   mix run problema2.exs datos/problema2/invalido  -> otro caso
#   mix run problema2.exs <entrada> <salida>        -> carpeta de salida propia

case System.argv() do
  [] -> Problema2.main()
  [entrada] -> Problema2.main(entrada)
  [entrada, salida | _] -> Problema2.main(entrada, salida)
end
```

ejecutar_todo.exs — 20 líneas

```elixir
# Ejecuta todos los casos de prueba de los dos problemas.
# Cada caso deja sus resultados en salidas/problemaN/<caso>/.
#
#   mix run ejecutar_todo.exs

for problema <- ["problema1", "problema2"] do
  casos =
    Path.join("datos", problema)
    |> File.ls!()
    |> Enum.sort()

  for caso <- casos do
    entrada = Path.join(["datos", problema, caso])

    case problema do
      "problema1" -> Problema1.main(entrada)
      "problema2" -> Problema2.main(entrada)
    end
  end
end
```

menu.exs — 54 líneas

```elixir
defmodule Menu do
  @moduledoc """
  Menú interactivo para la sustentación. Reutiliza los módulos del curso:

  - `Util.mostrar_mensaje/1` (versión 1) para el encabezado,
  - `Util2.ingresar/2` (versión 2) para leer y validar la opción y el caso.

      mix run menu.exs
  """

  def main do
    Util.mostrar_mensaje("""

    ================================================
     Parcial 2 - Programación III - Elixir
     1. Rutas de entrega en una red dirigida
     2. Regiones de un mapa de colores + SVG
     0. Salir
    ================================================
    """)

    case Util2.ingresar("Elija una opción: ", :entero) do
      0 ->
        Util2.mostrar("Hasta luego.", :mensaje)

      opcion when opcion in [1, 2] ->
        ejecutar(opcion)
        main()

      _ ->
        Util2.mostrar("Opción no válida.", :error)
        main()
    end
  end

```

```
  defp ejecutar(opcion) do
    carpeta = "datos/problema#{opcion}"
    casos = carpeta |> File.ls!() |> Enum.sort()

    casos
    |> Util2.convertir_coleccion_mensaje()
    |> then(&Util.mostrar_mensaje(["Casos disponibles:\n" | &1]))

    caso = Util2.ingresar("Nombre del caso: ", :texto)
    entrada = Path.join(carpeta, caso)

    case opcion do
      1 -> Problema1.main(entrada)
      2 -> Problema2.main(entrada)
    end
  end
end

Menu.main()
```

Módulos del curso (Util y Util2)

lib/util/util.ex — 107 líneas

```
defmodule Util do
  @moduledoc """
  Módulo con funciones que se reutilizan
  - autor: Julián E. Gutiérrez P.
  - fecha: Junio del 2026
  - licencia: GNU GPL v3
  """

  @doc """
  Función para mostrar un mensaje en la pantalla.

  ## Parámetro

  - mensaje: texto que se le presenta al usuario

  ## Ejemplo

  iex> Util.mostrar_mensaje("Hola Mundo")

  o puede usar

  "Hola Mundo" |> Util.mostrar_mensaje()
  """
  def mostrar_mensaje(mensaje) do
    mensaje
    |> IO.puts()
  end

  @doc """
  Función para mostrar un mensaje de error

  ## Parámetro

  - mensaje: texto que se le presenta al usuario

  ## Ejemplo

  iex> Util.mostrar_error("error ...")

  o puede usar

  "error ..." |> Util.mostrar_error()
  """
  def mostrar_error(mensaje) do
    IO.puts(:standard_error, mensaje)
  end

  @doc """
  Función para ingresar un dato desde teclado
```

```
50
51    ## Parámetro
52
53    - mensaje: texto que se le presenta al usuario
54    - tipo: identifica el tipo de dato.
55          :texto para ingresar una cadena
56          :entero para ingresar un valor entero
57          :real para ingresar un valor real
58
59    ## Ejemplo
60
61    iex> Util.ingresar("Ingresar nombre: ", :texto)
62    iex> Util.ingresar("Ingresar edad: ", :entero)
63    iex> Util.ingresar("Ingresar altura: ", :real)
64
65    o puede usar
66
67    "Ingresar nombre: " |> Util.ingresar(:texto)
68    "Ingresar edad: "   |> Util.ingresar(:entero)
69    "Ingresar altura: " |> Util.ingresar(:real)
70    """
71    def ingresar(mensaje, :texto) do
72      mensaje
73      |> IO.gets()
74      |> String.trim()
75    end
76
77    def ingresar(mensaje, :entero) do
78      ingresar(
79        mensaje,
80        &String.to_integer/1,
81        :entero
82      )
83    end
84
85    def ingresar(mensaje, :real) do
86      ingresar(
87        mensaje,
88        &String.to_float/1,
89        :real
90      )
91    end
92
93    defp ingresar(mensaje, parser, tipo_dato) do
94      try do
95        mensaje
96        |> Util.ingresar(:texto)
97        |> parser.()
98      rescue
99        ArgumentError ->
100         "Error, se espera que ingrese un número #{tipo_dato}\n"
101         |> mostrar_error()
102
103         mensaje
104         |> ingresar(tipo_dato)
105     end
106   end
107 end
```

lib/util/util2.ex — 339 líneas

```
1   defmodule Util2 do
2     @moduledoc """
3     Funciones auxiliares de propósito general para entrada y salida de datos,
4     procesamiento de colecciones y conversión de sus elementos a texto.
5
6     El módulo incluye funciones para:
7
8     - leer datos desde teclado;
9     - validar valores enteros, reales y booleanos;
10    - ingresar colecciones de elementos;
11    - mostrar mensajes y errores;
12    - ordenar y filtrar colecciones;
13    - convertir colecciones a una representación textual.
```

```
14
15  ## Información
16
17  - **Versión:** 2.0
18  - **Autor:** Julián E. Gutiérrez P.
19  - **Fecha:** septiembre de 2026
20  - **Licencia:** GNU GPL v3
21  """
22
23  # ----------------------------------
24  # Salida de datos
25  # ----------------------------------
26
27  @doc """
28  Muestra un mensaje en la salida indicada.
29
30  El segundo argumento determina si el contenido se envía a la salida
31  estándar o a la salida de errores.
32
33  ## Parámetros
34
35  - `mensaje`: contenido que se desea mostrar.
36  - `:mensaje`: muestra el contenido en la salida estándar.
37  - `:error`: muestra el contenido en la salida estándar de errores.
38
39  ## Ejemplos
40
41      Util2.mostrar("Hola Mundo", :mensaje)
42
43      Util2.mostrar("Operación incorrecta", :error)
44
45  También puede utilizarse mediante el operador `|>`:
46
47      "Hola Mundo"
48      |> Util2.mostrar(:mensaje)
49
50      "Operación incorrecta"
51      |> Util2.mostrar(:error)
52  """
53  def mostrar(mensaje, :mensaje), do: IO.puts(mensaje)
54  def mostrar(mensaje, :error), do: IO.puts(:standard_error, mensaje)
55
56  # ----------------------------------
57  # Entrada de datos
58  # ----------------------------------
59
60  @doc """
61  Lee desde teclado un dato o una colección de datos.
62
63  El comportamiento de la función depende del segundo argumento.
64
65  ## Entrada de un dato
66
67  Cuando el primer argumento es una pregunta, pueden utilizarse los
68  siguientes tipos:
69
70  - `:texto`: lee una cadena de caracteres.
71  - `:entero`: lee y valida un número entero.
72  - `:real`: lee y valida un número real.
73  - `:boolean`: lee una respuesta `s` o `n` y retorna `true` o `false`.
74
75  Por ejemplo:
76
77      Util2.ingresar("Ingrese el nombre: ", :texto)
78
79      Util2.ingresar("Ingrese la edad: ", :entero)
80
81      Util2.ingresar("Ingrese la altura: ", :real)
82
83      Util2.ingresar("¿Desea continuar (s/n)? ", :boolean)
84
85  También puede utilizarse mediante el operador `|>`:
86
```

```
    "Ingrese la edad: "
    |> Util2.ingresar(:entero)

## Entrada de una colección

Para ingresar una colección de elementos puede proporcionarse una
función de aridad cero encargada de leer cada elemento:

    Util2.ingresar(&Reserva.ingresar/0, :coleccion)

La función preguntará después de cada elemento si existen más datos.

También existen formas abreviadas para colecciones de tipos básicos:

- `:coleccion_textos`
- `:coleccion_enteros`
- `:coleccion_reales`

Por ejemplo:

    Util2.ingresar("Ingrese un número: ", :coleccion_enteros)

    Util2.ingresar("Ingrese un nombre: ", :coleccion_textos)
"""
def ingresar(pregunta, :coleccion_reales) do
  ingresar(fn -> ingresar(pregunta, :real) end, :coleccion)
end

def ingresar(pregunta, :coleccion_enteros) do
  ingresar(fn -> ingresar(pregunta, :entero) end, :coleccion)
end

def ingresar(pregunta, :coleccion_textos) do
  ingresar(fn -> ingresar(pregunta, :texto) end, :coleccion)
end

def ingresar(ingresar_elemento, :coleccion) do
  ingresar_coleccion(ingresar_elemento, [])
end

def ingresar(pregunta, :boolean) do
  ingresar(
    pregunta,
    fn texto ->
      case String.downcase(texto) do
        "s" -> {true, ""}
        "n" -> {false, ""}
        _ -> :error
      end
    end,
    :boolean
  )
end

def ingresar(pregunta, :real) do
  ingresar(pregunta, &Float.parse/1, :real)
end

def ingresar(pregunta, :entero) do
  ingresar(pregunta, &Integer.parse/1, :entero)
end

def ingresar(mensaje, :texto) do
  mensaje
  |> IO.gets()
  |> String.trim()
end

# ----------------------------------
# Operaciones sobre colecciones
# ----------------------------------

@doc """
```

```
  Ordena los elementos de una colección.

  Por defecto, los elementos se ordenan de forma ascendente utilizando
  como criterio el propio elemento.

  ## Parámetros

  - `coleccion`: colección que se desea ordenar.
  - `sentido`: orden `:asc` o `:desc`. Su valor por defecto es `:asc`.
  - `obtener_campo`: función que obtiene el valor utilizado como criterio
    de ordenamiento. Por defecto se utiliza `& &1`.

  La función criterio también puede retornar una tupla para ordenar por
  varios campos.

  ## Ejemplos

  Orden ascendente:

      Util2.ordenar(coleccion)

  Orden descendente:

      Util2.ordenar(coleccion, :desc)

  Si los elementos son estructuras con los campos `nombre` y `edad`,
  pueden ordenarse por edad:

      Util2.ordenar(coleccion, :asc, & &1.edad)

  Para ordenar primero por nombre y, en caso de empate, por edad:

      Util2.ordenar(coleccion, :asc, &{&1.nombre, &1.edad})
  """
  def ordenar(coleccion, sentido \\ :asc, obtener_campo \\ & &1) do
    Enum.sort_by(coleccion, obtener_campo, sentido)
  end

  @doc """
  Filtra una colección de cadenas conservando aquellas cuya longitud
  sea menor o igual al valor indicado.

  ## Parámetros

  - `coleccion`: colección de cadenas que se desea filtrar.
  - `longitud`: longitud máxima permitida.

  ## Ejemplo

  Obtiene las cadenas de longitud menor o igual a `3`:

      Util2.aplicar_filtro_longitud(coleccion, 3)
  """
  def aplicar_filtro_longitud(coleccion, longitud) do
    aplicar_filtro(coleccion, &(String.length(&1) <= longitud))
  end

  @doc """
  Filtra una colección de cadenas conservando aquellas que comienzan
  con una subcadena determinada.

  ## Parámetros

  - `coleccion`: colección de cadenas que se desea filtrar.
  - `inicia`: subcadena con la que deben comenzar los elementos seleccionados.

  ## Ejemplo

  Obtiene las cadenas que comienzan con `"Ana"`:

      Util2.aplicar_filtro_inicial(coleccion, "Ana")
  """
  def aplicar_filtro_inicial(coleccion, inicia) do
```

```elixir
  aplicar_filtro(coleccion, &String.starts_with?(&1, inicia))
end

@doc """
Filtra una colección de cadenas conservando aquellas que cumplan con
la función de filtro.

## Parámetros

- `coleccion`: colección de cadenas que se desea filtrar.
- `filtro`: función para filtrar los elementos.

## Ejemplo

Obtiene las cadenas que comienzan con `"Ana"`:

    Util2.aplicar_filtro(coleccion, &String.starts_with?(&1, "Ana"))
"""
def aplicar_filtro(coleccion, filtro) do
  Enum.filter(coleccion, filtro)
end

@doc """
Aplica un formato textual a cada elemento de una colección.

Por defecto, cada elemento se representa como un ítem de una lista
precedido por un guion.

## Parámetros

- `coleccion`: colección cuyos elementos se desean convertir.
- `formato`: función que recibe un elemento y retorna su representación
  textual. Por defecto:

      fn elemento -> " - \#{elemento}\\n" end

## Ejemplos

Con el formato predeterminado:

    Util2.convertir_coleccion_mensaje(coleccion)

produce una representación equivalente a:

    - Elemento 1
    - Elemento 2
    - Elemento 3

Para separar los elementos mediante tabuladores:

    Util2.convertir_coleccion_mensaje(
      coleccion,
      fn elemento -> "\#{elemento}\\t" end
    )
"""
def convertir_coleccion_mensaje(
      coleccion,
      formato \\ fn elemento -> " - #{elemento}\n" end
    ) do
  Enum.map(coleccion, formato)
end

# ----------------------------------
# Funciones auxiliares privadas
# ----------------------------------

# Ingresa recursivamente los elementos de una colección.
#
# `ingresar_elemento` es una función de aridad cero encargada de
# obtener cada elemento y `lista_actual` actúa como acumulador.
defp ingresar_coleccion(ingresar_elemento, lista_actual) do
  elemento = ingresar_elemento.()
  nueva_lista = [elemento | lista_actual]
```

```elixir

    case ingresar("\n¿Hay más datos (s/n)? ", :boolean) do
      true ->
        ingresar_coleccion(ingresar_elemento, nueva_lista)

      false ->
        Enum.reverse(nueva_lista)
    end
  end

  # Lee y valida un dato utilizando la función `parser`.
  #
  # Si el valor no puede interpretarse como el tipo solicitado,
  # muestra un mensaje de error y solicita nuevamente el dato.
  defp ingresar(pregunta, parser, tipo_dato) do
    resultado =
      pregunta
      |> ingresar(:texto)
      |> parser.()

    case resultado do
      {valor, ""} ->
        valor

      _ ->
        mostrar(
          "El valor ingresado no es válido para el tipo #{tipo_dato}. Intente nuevamente.\n",
          :error
        )

        ingresar(pregunta, tipo_dato)
    end
  end
end
```

Módulos comunes

lib/comun/entrada.ex — 84 líneas

```elixir
defmodule Entrada do
  @moduledoc """
  Lectura **perezosa** de los archivos de entrada. La usan los dos problemas.

  Cumple los requisitos generales 1, 2, 3 y 6 del parcial:

  - Lee cada archivo con `File.stream!/1`, línea por línea (nunca `File.read!/1`).
  - Limpia y filtra las líneas con transformaciones perezosas (`Stream`), sin
    crear listas intermedias.
  - Ignora las líneas en blanco y las que empiezan por `#` (comentarios).
  - Conserva el número de **línea física** original para reportar errores.
  - Verifica antes de leer que los archivos requeridos existan.
  """

  @doc """
  Verifica que existan todos los archivos requeridos.

  Retorna `:ok` o `{:error, faltantes}` con la lista de rutas que no existen.
  Así el programa informa cuál falta en lugar de terminar con una excepción.

  Reutiliza `Util2.aplicar_filtro/2` (módulo del curso) para quedarse con las
  rutas que no son archivos regulares.

  ## Ejemplo

      Entrada.verificar_archivos(["datos/tramos.csv", "datos/consultas.txt"])
  """
  def verificar_archivos(rutas) do
    case Util2.aplicar_filtro(rutas, &(not File.regular?(&1))) do
      [] -> :ok
      faltantes -> {:error, faltantes}
    end
  end

```

```
  @doc """
  Devuelve un **Stream** (perezoso) de tuplas `{numero_linea, texto}` con las
  líneas útiles del archivo.

  Hasta que alguien consuma el Stream no se lee nada del disco. Cada línea:

  1. se numera con su línea física (empezando en 1),
  2. se limpia (quita el salto de línea `\\r\\n`, espacios finales y un posible BOM),
  3. se descarta si está en blanco o es un comentario.

  ## Ejemplo

      "datos/problema1/ejemplo/tramos.csv"
      |> Entrada.lineas_utiles()
      |> Enum.take(2)
      #=> [{2, "A,B,8,5"}, {3, "B,D,7,4"}]
  """
  def lineas_utiles(ruta) do
    ruta
    |> File.stream!()
    |> Stream.with_index(1)
    |> Stream.map(&limpiar/1)
    |> Stream.reject(fn {_numero, texto} -> ignorable?(texto) end)
  end

  @doc """
  Indica si una línea se debe ignorar: está en blanco o su primer carácter no
  blanco es `#`.

  ## Ejemplos

      Entrada.ignorable?("   ")          #=> true
      Entrada.ignorable?("  # comentario") #=> true
      Entrada.ignorable?("A,B,8,5")      #=> false
  """
  def ignorable?(texto) do
    sin_espacios = String.trim(texto)
    sin_espacios == "" or String.starts_with?(sin_espacios, "#")
  end

  # Quita el BOM (si el archivo se guardó con él) y los espacios/saltos finales.
  defp limpiar({linea, numero}) do
    texto =
      linea
      |> String.trim_leading("")
      |> String.trim_trailing()

    {numero, texto}
  end
end
```

lib/comun/salida.ex — 73 líneas

```
defmodule Salida do
  @moduledoc """
  Escritura **por flujo** de los archivos de salida (requisito general 4).

  Nunca se arma un único string con todo el resultado: cada línea o fragmento
  se escribe en el archivo a medida que se produce. Cada ejecución crea o
  reemplaza el archivo (modo `:write`).

  Hay dos formas de escribir:

  - `escribir/2`: recibe cualquier enumerable de líneas (lista o `Stream`) y lo
    vuelca al archivo con `Stream.into/2`. Se usa para archivos que se generan
    de una sola vez (CSV, SVG, reportes).
  - `con_archivo/2` + `linea/2`: abre el archivo y entrega el dispositivo para
    escribir línea por línea mientras se procesa algo (por ejemplo, cada
    consulta del problema 1 en cuanto se resuelve).
  """

  @doc """
  Escribe en `ruta` cada elemento de `lineas` seguido de un salto de línea.

```

```
  `lineas` puede ser una lista o un `Stream`; si es un Stream, cada línea se
  calcula justo antes de escribirse.

  ## Ejemplo

      Salida.escribir("salidas/demo.txt", ["línea 1", "línea 2"])
  """
  def escribir(ruta, lineas) do
    preparar_carpeta(ruta)

    lineas
    |> Stream.map(&[&1, "\n"])
    |> Stream.into(File.stream!(ruta))
    |> Stream.run()

    ruta
  end

  @doc """
  Abre `ruta` para escritura (UTF-8), ejecuta `funcion.(dispositivo)` y cierra
  el archivo al terminar. Retorna lo que retorne `funcion`.

  ## Ejemplo

      Salida.con_archivo("salidas/demo.txt", fn archivo ->
        Salida.linea(archivo, "Hola")
      end)
  """
  def con_archivo(ruta, funcion) do
    preparar_carpeta(ruta)

    case File.open(ruta, [:write, :utf8], funcion) do
      {:ok, resultado} ->
        resultado

      {:error, razon} ->
        Util2.mostrar("No se pudo escribir #{ruta}: #{inspect(razon)}", :error)
        {:error, razon}
    end
  end

  @doc "Escribe una línea (con salto de línea) en un archivo abierto con `con_archivo/2`."
  def linea(dispositivo, texto), do: IO.write(dispositivo, [texto, "\n"])

  @doc """
  Elimina archivos de salida de una ejecución anterior. Se usa cuando la
  validación falla, para que no queden resultados viejos que confundan.
  """
  def eliminar(rutas), do: Enum.each(rutas, &File.rm/1)

  defp preparar_carpeta(ruta), do: File.mkdir_p!(Path.dirname(ruta))
end
```

lib/comun/lista.ex — 37 líneas

```
defmodule Lista do
  @moduledoc """
  Funciones **recursivas propias** sobre listas.

  El requisito general 7 prohíbe usar los módulos de colecciones y flujos de
  la biblioteca estándar dentro de las validaciones previas. Por eso las
  validaciones de ambos problemas usan estas funciones, escritas con
  recursión de cola y un acumulador.
  """

  @doc """
  Invierte una lista.

  Las validaciones acumulan resultados con `[nuevo | acumulador]` (agregar al
  inicio es O(1)), lo que los deja al revés; al final se invierten una vez.

  ## Ejemplo

      Lista.invertir([1, 2, 3]) #=> [3, 2, 1]
```

```
  """
  def invertir(lista), do: invertir(lista, [])

  defp invertir([], acumulador), do: acumulador
  defp invertir([cabeza | cola], acumulador), do: invertir(cola, [cabeza | acumulador])

  @doc """
  Cuenta los elementos de una lista.

  ## Ejemplo

      Lista.contar(["A", "B", "."]) #=> 3
  """
  def contar(lista), do: contar(lista, 0)

  defp contar([], total), do: total
  defp contar([_cabeza | cola], total), do: contar(cola, total + 1)
end
```

Problema 1: rutas de entrega

lib/problema1/tramo.ex — 103 líneas

```
defmodule Tramo do
  @moduledoc """
  Un **tramo** dirigido de la red de distribución: va de `origen` a `destino`,
  tarda `minutos` y cuesta `costo`.

  Además de los cuatro datos del tramo, el struct conserva de dónde salió:

  - `linea`: la línea física de `tramos.csv`;
  - `contenido`: el texto original de esa línea, tal como estaba escrito.

  Así la depuración puede reportar la línea física, el contenido y el motivo
  de cada descarte, como pide el enunciado.
  """

  defstruct origen: "", destino: "", minutos: 0, costo: 0, linea: 0, contenido: ""

  @doc """
  Convierte una línea de `tramos.csv` en un `%Tramo{}`.

  Acepta el texto solo o la tupla `{numero_linea, texto}` que produce
  `Entrada.lineas_utiles/1`; en ese caso el número queda guardado en el struct.
  En los dos casos el texto original queda en el campo `contenido`.

  Retorna `{:ok, tramo}` o `{:error, motivo}`.

  ## Ejemplos

      Tramo.parse("A,B,8,5")
      #=> {:ok, %Tramo{origen: "A", destino: "B", minutos: 8, costo: 5, linea: 0, contenido: "A,B,8,5"}}

      Tramo.parse({2, "A, B, 8, 5"})
      #=> {:ok, %Tramo{origen: "A", destino: "B", minutos: 8, costo: 5, linea: 2, contenido: "A, B, 8, 5"}}

      Tramo.parse("A,B,ocho,5")
      #=> {:error, "minutos: 'ocho' no es un número entero"}
  """
  def parse({numero_linea, texto}) do
    case parse(texto) do
      {:ok, %Tramo{} = tramo} -> {:ok, %Tramo{tramo | linea: numero_linea}}
      error -> error
    end
  end

  def parse(texto) when is_binary(texto) do
    campos =
      texto
      |> String.split(",")
      |> Enum.map(&String.trim/1)

    case campos do
```

```
      [origen, destino, minutos, costo] ->
        with :ok <- validar_centro(origen, "origen"),
             :ok <- validar_centro(destino, "destino"),
             {:ok, minutos} <- entero_positivo(minutos, "minutos"),
             {:ok, costo} <- entero_positivo(costo, "costo") do
          {:ok,
           %Tramo{
             origen: origen,
             destino: destino,
             minutos: minutos,
             costo: costo,
             contenido: texto
           }}
        end

      _ ->
        {:error, "se esperaban 4 campos (origen,destino,minutos,costo) y hay #{length(campos)}"}
    end
  end

  @doc """
  Descripción corta del tramo para los reportes.

  ## Ejemplo

      Tramo.describir(%Tramo{origen: "A", destino: "B", minutos: 8, costo: 5})
      #=> "A→B (8 min, costo 5)"
  """
  def describir(%Tramo{origen: o, destino: d, minutos: m, costo: c}) do
    "#{o}→#{d} (#{m} min, costo #{c})"
  end

  # --- Validaciones de cada campo ---------------------------------------

  defp validar_centro("", campo), do: {:error, "el #{campo} está vacío"}
  defp validar_centro(_centro, _campo), do: :ok

  # Integer.parse/1 devuelve {numero, resto}; solo es válido si no sobra texto.
  defp entero_positivo(texto, campo) do
    case Integer.parse(texto) do
      {numero, ""} when numero > 0 -> {:ok, numero}
      {numero, ""} -> {:error, "#{campo} debe ser un entero positivo y es #{numero}"}
      _ -> {:error, "#{campo}: '#{texto}' no es un número entero"}
    end
  end

  # Permite usar "#{tramo}" o to_string(tramo): reconstruye la línea CSV.
  defimpl String.Chars do
    def to_string(%Tramo{origen: o, destino: d, minutos: m, costo: c}) do
      "#{o},#{d},#{m},#{c}"
    end
  end
end
```

lib/problema1/itinerario.ex — 49 líneas

```
defmodule Itinerario do
  @moduledoc """
  Un **itinerario simple**: la secuencia de `centros` recorridos (sin
  repetir), los `tramos` usados en orden, y los totales de `minutos` y `costo`.
  """

  defstruct centros: [], tramos: [], minutos: 0, costo: 0

  @doc """
  Construye un itinerario a partir del centro de origen y la lista de tramos
  recorridos, en orden. Suma tiempos y costos.

  ## Ejemplo

      Itinerario.desde_tramos("A", [tramo_a_b, tramo_b_d])
      #=> %Itinerario{centros: ["A", "B", "D"], minutos: 15, costo: 9, ...}
  """
  def desde_tramos(origen, tramos) do
```

```
    %Itinerario{
      centros: [origen | Enum.map(tramos, & &1.destino)],
      tramos: tramos,
      minutos: tramos |> Enum.map(& &1.minutos) |> Enum.sum(),
      costo: tramos |> Enum.map(& &1.costo) |> Enum.sum()
    }
  end

  @doc "Número de tramos del itinerario (criterio (a) del enunciado)."
  def numero_tramos(%Itinerario{tramos: tramos}), do: length(tramos)

  @doc ~S'Secuencia de centros, por ejemplo `"A → B → D"`.'
  def ruta(%Itinerario{centros: centros}), do: Enum.join(centros, " → ")

  @doc "Lista legible de los tramos usados, con su tiempo y costo."
  def detalle_tramos(%Itinerario{tramos: []}), do: "ninguno (origen y destino coinciden)"

  def detalle_tramos(%Itinerario{tramos: tramos}) do
    tramos
    |> Enum.map(&Tramo.describir/1)
    |> Enum.join(", ")
  end

  # "#{itinerario}" produce la línea del reporte, igual al formato del enunciado.
  defimpl String.Chars do
    def to_string(itinerario) do
      "#{Itinerario.ruta(itinerario)} | tramos: #{Itinerario.numero_tramos(itinerario)}" <>
        " | #{itinerario.minutos} min | costo: #{itinerario.costo}"
    end
  end
end
```

lib/problema1/depuracion.ex — 62 líneas

```
defmodule Rutas.Depuracion do
  @moduledoc """
  **Validación previa con recursión: depurar la red** (problema 1).

  Antes de construir el grafo se recorren recursivamente los tramos válidos y
  se descartan:

  1. los **bucles**: tramos que salen y llegan al mismo centro (`A,A,...`);
  2. las **repeticiones** de la misma pareja en la misma dirección. Si cambian
     el tiempo o el costo, se conserva la **primera** aparición.

  Los tramos en dirección contraria (`A→B` y `B→A`) son distintos y se
  conservan los dos.

  Requisito 7: estas funciones son recursivas propias y no usan los módulos
  de colecciones ni de flujos. Solo usan `Map` como tabla de "ya visto" y
  `Lista.invertir/1` (también recursiva) para devolver el orden original.
  """

  @doc """
  Depura la lista de tramos válidos.

  Retorna `{conservados, descartes}`, donde `descartes` es una lista de
  `{tramo, motivo}`. Ambas listas mantienen el orden del archivo, y cada tramo
  conserva su número de línea física en `tramo.linea`.

  ## Ejemplo

      Rutas.Depuracion.depurar([ab_linea2, aa_linea8, ab_linea9])
      #=> {[ab_linea2],
      #    [{aa_linea8, "bucle: ..."}, {ab_linea9, "repetido: ..."}]}
  """
  def depurar(tramos), do: depurar(tramos, %{}, [], [])

  # Caso base: no quedan tramos. Se invierten los acumuladores.
  defp depurar([], _vistos, conservados, descartes) do
    {Lista.invertir(conservados), Lista.invertir(descartes)}
  end

  # Bucle: el patrón exige que origen y destino sean el MISMO valor `c`.
```

```elixir
  defp depurar([%Tramo{origen: c, destino: c} = tramo | resto], vistos, conservados, descartes) do
    motivo = "bucle: el tramo sale y llega al mismo centro (#{c})"
    depurar(resto, vistos, conservados, [{tramo, motivo} | descartes])
  end

  # Tramo normal: se conserva solo si la pareja (origen, destino) no apareció antes.
  defp depurar([tramo | resto], vistos, conservados, descartes) do
    pareja = {tramo.origen, tramo.destino}

    case Map.fetch(vistos, pareja) do
      {:ok, linea_original} ->
        motivo =
          "repetido: la pareja #{tramo.origen} → #{tramo.destino} ya se definió en la " <>
            "línea #{linea_original}; se conserva la primera aparición"

        depurar(resto, vistos, conservados, [{tramo, motivo} | descartes])

      :error ->
        depurar(resto, Map.put(vistos, pareja, tramo.linea), [tramo | conservados], descartes)
    end
  end
end
```

lib/problema1/busqueda.ex — 106 líneas

```elixir
defmodule Rutas.Busqueda do
  @moduledoc """
  Construcción del grafo dirigido y **búsqueda recursiva** de todos los
  itinerarios simples entre dos centros (problema 1).

  El grafo es un mapa `%{centro => [tramos que salen de ese centro]}`. La
  lista de cada centro conserva el orden del archivo, y por eso los
  itinerarios salen en el mismo orden del enunciado.
  """

  @doc """
  Arma el grafo (lista de adyacencia) a partir de los tramos depurados.

  ## Ejemplo

      Rutas.Busqueda.construir_grafo(tramos)
      #=> %{"A" => [A→B, A→C], "B" => [B→D, B→C], "C" => [C→D, C→B]}
  """
  def construir_grafo(tramos), do: Enum.group_by(tramos, & &1.origen)

  @doc "Conjunto (`MapSet`) de todos los centros que aparecen en la red."
  def centros(tramos) do
    tramos
    |> Enum.flat_map(&[&1.origen, &1.destino])
    |> MapSet.new()
  end

  @doc """
  Encuentra **todos** los itinerarios simples de `origen` a `destino`.

  Es un recorrido en profundidad con *backtracking*: desde el centro actual se
  prueba cada tramo de salida cuyo destino no se haya visitado. Como los datos
  son inmutables, "deshacer" una elección es automático: cada llamada
  recursiva recibe su propia copia del conjunto `visitados`.

  ## Ejemplo (red del enunciado)

      Rutas.Busqueda.itinerarios(grafo, "A", "D") |> Enum.map(&Itinerario.ruta/1)
      #=> ["A → B → D", "A → B → C → D", "A → C → D", "A → C → B → D"]
  """
  def itinerarios(grafo, origen, destino) do
    grafo
    |> explorar(origen, destino, MapSet.new([origen]), [])
    |> Enum.map(&Itinerario.desde_tramos(origen, &1))
  end

  # Caso base: llegamos al destino. El camino se acumuló al revés.
  defp explorar(_grafo, destino, destino, _visitados, camino_al_reves) do
    [Enum.reverse(camino_al_reves)]
```

```elixir
  end

  # Caso recursivo: se prueba cada tramo de salida hacia un centro no visitado.
  # Si un centro no tiene salidas, Map.get devuelve [] y la rama no aporta caminos.
  defp explorar(grafo, actual, destino, visitados, camino_al_reves) do
    grafo
    |> Map.get(actual, [])
    |> Enum.reject(&MapSet.member?(visitados, &1.destino))
    |> Enum.flat_map(fn tramo ->
      explorar(
        grafo,
        tramo.destino,
        destino,
        MapSet.put(visitados, tramo.destino),
        [tramo | camino_al_reves]
      )
    end)
  end

  @doc """
  Para una lista de itinerarios, retorna los **números** (empezando en 1) de
  los que minimizan cada criterio, conservando todos los empates:

  - `:tramos`: menor número de tramos,
  - `:minutos`: menor tiempo,
  - `:costo`: menor costo.

  ## Ejemplo (red del enunciado, consulta A → D)

      Rutas.Busqueda.optimos(itinerarios)
      #=> %{tramos: [1, 3], minutos: [1], costo: [3]}
  """
  def optimos([]), do: %{tramos: [], minutos: [], costo: []}

  def optimos(itinerarios) do
    numerados = Enum.with_index(itinerarios, 1)

    %{
      tramos: mejores(numerados, &Itinerario.numero_tramos/1),
      minutos: mejores(numerados, & &1.minutos),
      costo: mejores(numerados, & &1.costo)
    }
  end

  # Calcula el mínimo del criterio y filtra todos los itinerarios que lo alcanzan
  # (así se conservan los empates). Reutiliza Util2.aplicar_filtro/2 del curso.
  defp mejores(numerados, criterio) do
    minimo =
      numerados
      |> Enum.map(fn {itinerario, _numero} -> criterio.(itinerario) end)
      |> Enum.min()

    numerados
    |> Util2.aplicar_filtro(fn {itinerario, _numero} -> criterio.(itinerario) == minimo end)
    |> Enum.map(fn {_itinerario, numero} -> numero end)
  end
end
```

lib/problema1/alcance.ex — 94 líneas

```elixir
defmodule Rutas.Alcance do
  @moduledoc """
  **Stream infinito de centros alcanzables** (problema 1).

  - Paso 0: solo el origen.
  - Paso k+1: el conjunto del paso k más todos los destinos de los tramos que
    salen de los centros de ese conjunto.

  ## ¿Por qué converge si el Stream es infinito?

  1. Los conjuntos **solo crecen**: el paso k+1 contiene al paso k (se hace
     una unión).
  2. Están **acotados**: nunca pueden tener más centros que la red, que es finita.

```

```
Una sucesión que crece y está acotada no puede crecer para siempre: después
de, como mucho, *n* pasos (con *n* el número de centros) un paso repite el
conjunto anterior. Desde ese momento todos los pasos son iguales (punto
fijo), porque expandir el mismo conjunto da el mismo resultado.

El Stream (`Stream.iterate/2`) **describe** la sucesión infinita, pero es
perezoso: no calcula ningún paso hasta que se le pide. `Enum.reduce_while/3`
pide pasos uno a uno y se detiene (`:halt`) en el primer estado repetido,
así que solo se calculan los pasos necesarios.
"""

@doc """
El Stream infinito de conjuntos alcanzables desde `origen`.

## Ejemplo (red del enunciado)

    Rutas.Alcance.secuencia(grafo, "A") |> Enum.take(4)
    #=> [MapSet<["A"]>, MapSet<["A","B","C"]>, MapSet<["A","B","C","D"]>, MapSet<["A","B","C","D"]>]
"""
def secuencia(grafo, origen) do
  Stream.iterate(MapSet.new([origen]), &expandir(&1, grafo))
end

@doc """
Un paso de la sucesión: agrega al conjunto los destinos de todos los tramos
que salen de sus centros.
"""
def expandir(conjunto, grafo) do
  conjunto
  |> Enum.flat_map(&Map.get(grafo, &1, []))
  |> Enum.map(& &1.destino)
  |> MapSet.new()
  |> MapSet.union(conjunto)
end

@doc """
Consume el Stream infinito hasta el primer estado repetido consecutivo.

Retorna `{estados_distintos, paso_estable}`:

- `estados_distintos`: los conjuntos de los pasos 0..m, en orden.
- `paso_estable`: el paso m donde se estabiliza (el paso m+1 lo repite).

Solo se calculan m+2 elementos del Stream (pasos 0..m+1).

## Ejemplo (red del enunciado)

    Rutas.Alcance.hasta_estabilizar(grafo, "A")
    #=> {[MapSet<["A"]>, MapSet<["A","B","C"]>, MapSet<["A","B","C","D"]>], 2}
"""
def hasta_estabilizar(grafo, origen) do
  grafo
  |> secuencia(origen)
  |> Stream.with_index()
  |> Enum.reduce_while([], &comparar_con_anterior/2)
end

# Primer elemento: aún no hay con qué comparar.
defp comparar_con_anterior({estado, _paso}, []), do: {:cont, [estado]}

# El acumulador guarda los estados al revés: el anterior es la cabeza.
defp comparar_con_anterior({estado, paso}, [anterior | _] = estados) do
  if MapSet.equal?(estado, anterior) do
    {:halt, {Enum.reverse(estados), paso - 1}}
  else
    {:cont, [estado | estados]}
  end
end

@doc ~S'Representación ordenada de un conjunto, por ejemplo `"{A, B, C}"`.'
def formatear(conjunto) do
  texto =
    conjunto
```

```
      |> MapSet.to_list()
      |> Util2.ordenar()
      |> Enum.join(", ")

    "{#{texto}}"
  end
end
```

lib/problema1/problema1.ex 262 líneas

```elixir
defmodule Problema1 do
  @moduledoc """
  **Problema 1: rutas de entrega en una red dirigida.** Programa principal.

  Flujo completo (descomposición del problema):

  1. Verificar que existan `tramos.csv` y `consultas.txt`.
  2. Leer `tramos.csv` de forma perezosa y convertir cada línea con
     `Tramo.parse/1` → errores a `errores_tramos.txt`.
  3. Depurar recursivamente (`Rutas.Depuracion`) → `depuracion_tramos.txt`.
  4. Construir el grafo (`Rutas.Busqueda.construir_grafo/1`).
  5. Para cada consulta (leída también como Stream, una a la vez):
     - si un centro no existe, se informa y no se busca;
     - Stream infinito de alcance (`Rutas.Alcance`) → `alcance_centros.txt`;
     - si el destino no es alcanzable: «sin itinerarios», sin búsqueda;
     - si lo es: búsqueda recursiva y óptimos → `itinerarios.txt`.
  6. En pantalla, el total de itinerarios por consulta.

  ## Uso

      mix run problema1.exs datos/problema1/ejemplo

  o desde `iex -S mix`:

      Problema1.main("datos/problema1/ejemplo", "salidas/problema1/ejemplo")
  """

  alias Rutas.{Alcance, Busqueda, Depuracion}

  @entrada_por_defecto "datos/problema1/ejemplo"

  @doc """
  Ejecuta el problema 1 con los archivos de `dir_entrada` y deja los resultados
  en `dir_salida` (por defecto `salidas/problema1/<nombre del caso>`).

  Retorna `{:ok, totales}` con `[{"A,D", 4}, ...]` o `{:error, motivo}`.
  Nunca termina con una excepción por un archivo faltante.
  """
  def main(dir_entrada \\ @entrada_por_defecto, dir_salida \\ nil) do
    dir_salida = dir_salida || Path.join("salidas/problema1", Path.basename(dir_entrada))
    ruta_tramos = Path.join(dir_entrada, "tramos.csv")
    ruta_consultas = Path.join(dir_entrada, "consultas.txt")

    Util2.mostrar("\n=== Problema 1: rutas de entrega (#{dir_entrada}) ===", :mensaje)

    case Entrada.verificar_archivos([ruta_tramos, ruta_consultas]) do
      :ok ->
        ejecutar(ruta_tramos, ruta_consultas, dir_salida)

      {:error, faltantes} ->
        Enum.each(faltantes, &Util2.mostrar("Falta el archivo requerido: #{&1}", :error))
        {:error, {:faltan_archivos, faltantes}}
    end
  end

  # ---------------------------------------------------------------------------
  # Etapas
  # ---------------------------------------------------------------------------

  defp ejecutar(ruta_tramos, ruta_consultas, dir_salida) do
    # 2. Lectura perezosa + parse. El grafo se arma DESPUÉS de consumir el Stream.
    {validos, errores} = leer_tramos(ruta_tramos)
    Salida.escribir(Path.join(dir_salida, "errores_tramos.txt"), lineas_errores(errores))

```

```elixir
    # 3. Validación previa recursiva.
    {conservados, descartes} = Depuracion.depurar(validos)
    Salida.escribir(Path.join(dir_salida, "depuracion_tramos.txt"), lineas_depuracion(descartes))

    # 4. Grafo.
    grafo = Busqueda.construir_grafo(conservados)
    centros = Busqueda.centros(conservados)

    Util2.mostrar(
      "Tramos válidos: #{length(validos)} | con error: #{length(errores)} | " <>
        "descartados al depurar: #{length(descartes)} | en la red: #{length(conservados)}",
      :mensaje
    )

    # 5 y 6. Consultas.
    totales = procesar_consultas(ruta_consultas, grafo, centros, dir_salida)
    Util2.mostrar("Archivos generados en #{dir_salida}", :mensaje)
    {:ok, totales}
  end

  # Consume el Stream de líneas UNA vez, separando tramos válidos y errores.
  # Los acumuladores se llenan al revés y se invierten al final.
  defp leer_tramos(ruta) do
    {validos, errores} =
      ruta
      |> Entrada.lineas_utiles()
      |> Stream.map(fn {numero, texto} -> {numero, texto, Tramo.parse({numero, texto})} end)
      |> Enum.reduce({[], []}, fn
        {_numero, _texto, {:ok, tramo}}, {validos, errores} ->
          {[tramo | validos], errores}

        {numero, texto, {:error, motivo}}, {validos, errores} ->
          {validos, [{numero, texto, motivo} | errores]}
      end)

    {Enum.reverse(validos), Enum.reverse(errores)}
  end

  # Las consultas se leen como Stream y se resuelven una por una: cada bloque se
  # escribe en los dos archivos de salida en cuanto la consulta se resuelve.
  defp procesar_consultas(ruta_consultas, grafo, centros, dir_salida) do
    Salida.con_archivo(Path.join(dir_salida, "itinerarios.txt"), fn itinerarios ->
      Salida.con_archivo(Path.join(dir_salida, "alcance_centros.txt"), fn alcance ->
        ruta_consultas
        |> Entrada.lineas_utiles()
        |> Stream.map(&parse_consulta/1)
        |> Enum.map(&resolver(&1, grafo, centros, itinerarios, alcance))
      end)
    end)
  end

  defp parse_consulta({numero, texto}) do
    case texto |> String.split(",") |> Enum.map(&String.trim/1) do
      [origen, destino] when origen != "" and destino != "" ->
        {:ok, origen, destino}

      _ ->
        {:error, numero, texto, "se esperaba el formato origen,destino"}
    end
  end

  # ---------------------------------------------------------------------------
  # Resolución de una consulta (pattern matching sobre el resultado del parse)
  # ---------------------------------------------------------------------------

  defp resolver({:error, numero, texto, motivo}, _grafo, _centros, itinerarios, alcance) do
    titulo = "=== Consulta inválida (línea #{numero}): #{texto} ==="
    escribir_bloque(itinerarios, [titulo, "Motivo: #{motivo}"])
    escribir_bloque(alcance, [titulo, "Motivo: #{motivo}"])
    Util2.mostrar("Línea #{numero}: consulta inválida '#{texto}' (#{motivo})", :error)
    {texto, :invalida}
  end
```

```elixir
defp resolver({:ok, origen, destino}, grafo, centros, itinerarios, alcance) do
  titulo = "=== #{origen} → #{destino} ==="
  consulta = "#{origen},#{destino}"

  case centros_inexistentes([origen, destino], centros) do
    [] ->
      resolver_existente(origen, destino, grafo, titulo, itinerarios, alcance, consulta)

    faltantes ->
      mensaje = mensaje_inexistentes(faltantes)
      escribir_bloque(itinerarios, [titulo, mensaje])
      escribir_bloque(alcance, [titulo, mensaje])
      Util2.mostrar("#{origen} → #{destino}: #{mensaje}", :error)
      {consulta, :centro_inexistente}
  end
end

defp resolver_existente(origen, destino, grafo, titulo, itinerarios, alcance, consulta) do
  # Stream infinito: se consumen solo los pasos necesarios.
  {estados, paso_estable} = Alcance.hasta_estabilizar(grafo, origen)
  alcanzable = MapSet.member?(List.last(estados), destino)
  escribir_bloque(alcance, lineas_alcance(titulo, estados, paso_estable, destino, alcanzable))

  if alcanzable do
    encontrados = Busqueda.itinerarios(grafo, origen, destino)
    escribir_bloque(itinerarios, lineas_itinerarios(titulo, encontrados))
    Util2.mostrar("#{origen} → #{destino}: #{total(length(encontrados))}", :mensaje)
    {consulta, length(encontrados)}
  else
    escribir_bloque(itinerarios, [
      titulo,
      "El destino #{destino} no es alcanzable desde #{origen}: sin itinerarios " <>
        "(se omitió la búsqueda exhaustiva).",
      "Total: 0 itinerarios"
    ])

    Util2.mostrar("#{origen} → #{destino}: sin itinerarios", :mensaje)
    {consulta, 0}
  end
end

defp centros_inexistentes(consultados, centros) do
  consultados
  |> Enum.uniq()
  |> Util2.aplicar_filtro(&(not MapSet.member?(centros, &1)))
end

defp mensaje_inexistentes([centro]),
  do: "el centro #{centro} no existe en la red; no se inicia la búsqueda"

defp mensaje_inexistentes(centros),
  do: "los centros #{Enum.join(centros, " y ")} no existen en la red; no se inicia la búsqueda"

# ------------------------------------------------------------------------
# Formato de los reportes (funciones puras: datos → líneas de texto)
# ------------------------------------------------------------------------

defp lineas_errores([]), do: ["Sin errores: todas las líneas de tramos.csv son correctas."]

defp lineas_errores(errores) do
  ["Líneas de tramos.csv con formato o datos incorrectos: #{length(errores)}"] ++
    Util2.convertir_coleccion_mensaje(errores, fn {numero, texto, motivo} ->
      "Línea #{numero}: #{texto} | motivo: #{motivo}"
    end)
end

defp lineas_depuracion([]), do: ["Sin descartes: la red no tiene bucles ni tramos repetidos."]

defp lineas_depuracion(descartes) do
  ["Tramos descartados en la depuración: #{length(descartes)}"] ++
    Util2.convertir_coleccion_mensaje(descartes, fn {tramo, motivo} ->
      "Línea #{tramo.linea}: #{tramo.contenido} | motivo: #{motivo}"
    end)
```

```elixir
  end

  defp lineas_alcance(titulo, estados, paso_estable, destino, alcanzable) do
    pasos =
      estados
      |> Enum.with_index()
      |> Enum.map(fn {conjunto, paso} -> "Paso #{paso}: #{Alcance.formatear(conjunto)}" end)

    [titulo] ++
      pasos ++
      [
        "Paso #{paso_estable + 1}: repite el paso #{paso_estable} → se estabiliza en el paso #{paso_estable}",
        "Elementos del Stream consumidos: #{paso_estable + 2} (pasos 0 a #{paso_estable + 1})",
        if(alcanzable,
          do: "Destino #{destino}: alcanzable",
          else: "Destino #{destino}: no alcanzable → sin itinerarios"
        )
      ]
  end

  defp lineas_itinerarios(titulo, encontrados) do
    optimos = Busqueda.optimos(encontrados)

    detalle =
      encontrados
      |> Enum.with_index(1)
      |> Enum.flat_map(fn {itinerario, numero} ->
        [
          "Itinerario #{numero}: #{itinerario}",
          "    tramos usados: #{Itinerario.detalle_tramos(itinerario)}"
        ]
      end)

    [titulo] ++
      detalle ++
      [
        "Total: #{total(length(encontrados))}",
        "Menos tramos: #{Enum.join(optimos.tramos, ", ")}",
        "Menor tiempo: #{Enum.join(optimos.minutos, ", ")}",
        "Menor costo: #{Enum.join(optimos.costo, ", ")}"
      ]
  end

  defp total(1), do: "1 itinerario"
  defp total(n), do: "#{n} itinerarios"

  # Escribe las líneas de un bloque y una línea en blanco de separación.
  defp escribir_bloque(dispositivo, lineas) do
    Enum.each(lineas, &Salida.linea(dispositivo, &1))
    Salida.linea(dispositivo, "")
  end
end
```

Problema 2: regiones y SVG

lib/problema2/color.ex — 49 líneas

```elixir
defmodule Color do
  @moduledoc """
  Un color de la paleta (`paleta.csv`): el `simbolo` del mapa, su `nombre`
  y el color `hex` con el que se dibuja. `linea` es la línea física de origen.
  """

  defstruct simbolo: "", nombre: "", hex: "", linea: 0

  @doc """
  Convierte una línea de `paleta.csv` (`simbolo,nombre,color_hex`) en un
  `%Color{}`. Recibe la tupla `{numero_linea, texto}`.

  Aquí solo se revisa la **forma** (3 campos, símbolo no vacío). El formato
  `#RRGGBB`, los repetidos y la existencia de `.` los revisa la validación
  recursiva (`Mapa.Validacion`).

```

```
  ## Ejemplos

      Color.parse({1, "A,agua,#2563EB"})
      #=> {:ok, %Color{simbolo: "A", nombre: "agua", hex: "#2563EB", linea: 1}}

      Color.parse({2, "A,agua"})
      #=> {:error, "se esperaban 3 campos (simbolo,nombre,color_hex) y hay 2"}
  """
  def parse({numero_linea, texto}) do
    case texto |> String.split(",") |> Enum.map(&String.trim/1) do
      ["", _nombre, _hex] ->
        {:error, "el símbolo está vacío"}

      [simbolo, nombre, hex] ->
        {:ok, %Color{simbolo: simbolo, nombre: nombre, hex: hex, linea: numero_linea}}

      campos ->
        {:error, "se esperaban 3 campos (simbolo,nombre,color_hex) y hay #{length(campos)}"}
    end
  end

  @doc """
  Construye la tabla `%{simbolo => %Color{}}` a partir de las líneas ya
  parseadas. Si un símbolo se repite, se queda la primera aparición.
  Se usa solo cuando la validación pasó.
  """
  def tabla(colores_parseados) do
    Enum.reduce(colores_parseados, %{}, fn
      {_linea, {:ok, color}}, tabla -> Map.put_new(tabla, color.simbolo, color)
      {_linea, {:error, _motivo}}, tabla -> tabla
    end)
  end
end
```

lib/problema2/region.ex — 61 líneas

```
defmodule Region do
  @moduledoc """
  Una **región**: grupo de casillas con el mismo símbolo conectadas en
  horizontal o vertical.

  - `id`: número asignado según la primera casilla encontrada (orden por filas).
  - `simbolo`: símbolo del mapa.
  - `celdas`: lista ordenada de coordenadas `{fila, columna}` (desde 0).
  - `area`: cantidad de casillas.
  - `perimetro`: lados que tocan el borde o una casilla de otro símbolo.
  - `desde` / `hasta`: esquinas de la caja envolvente.
  """

  defstruct id: 0, simbolo: "", celdas: [], area: 0, perimetro: 0, desde: {0, 0}, hasta: {0, 0}

  @doc """
  Crea la región a partir de sus casillas y calcula área, perímetro y caja.

  `celdas_mapa` es el mapa `%{{fila, columna} => simbolo}` completo.
  """
  def nueva(id, simbolo, celdas, celdas_mapa) do
    ordenadas = Enum.sort(celdas)
    filas = Enum.map(ordenadas, fn {fila, _columna} -> fila end)
    columnas = Enum.map(ordenadas, fn {_fila, columna} -> columna end)

    %Region{
      id: id,
      simbolo: simbolo,
      celdas: ordenadas,
      area: length(ordenadas),
      perimetro: perimetro(ordenadas, simbolo, celdas_mapa),
      desde: {Enum.min(filas), Enum.min(columnas)},
      hasta: {Enum.max(filas), Enum.max(columnas)}
    }
  end

  @doc """
  Las 4 casillas vecinas (arriba, abajo, izquierda, derecha). Las diagonales
```

```
  no cuentan. Puede devolver coordenadas fuera del mapa; quien la usa lo tiene
  en cuenta (`Map.get` da `nil` fuera del mapa).
  """
  def vecinos({fila, columna}) do
    [{fila - 1, columna}, {fila + 1, columna}, {fila, columna - 1}, {fila, columna + 1}]
  end

  # Cada lado de una casilla suma 1 si del otro lado hay borde (nil) o un símbolo
  # distinto (incluido el fondo ".").
  defp perimetro(celdas, simbolo, celdas_mapa) do
    celdas
    |> Enum.flat_map(&vecinos/1)
    |> Enum.count(&(Map.get(celdas_mapa, &1) != simbolo))
  end

  @doc ~S'Línea del CSV de salida: `"1;A;3;8;{0,0};{1,1}"`.'
  def a_csv(%Region{} = r) do
    "#{r.id};#{r.simbolo};#{r.area};#{r.perimetro};#{coordenada(r.desde)};#{coordenada(r.hasta)}"
  end

  @doc ~S'Coordenada con el formato del enunciado: `"{2,0}"`.'
  def coordenada({fila, columna}), do: "{#{fila},#{columna}}"
end
```

lib/problema2/validacion.ex — 159 líneas

```
defmodule Mapa.Validacion do
  @moduledoc """
  **Validación previa con recursión: mapa y paleta** (problema 2).

  Revisa, con funciones recursivas propias (requisito 7: sin los módulos de
  colecciones ni de flujos):

  - Paleta: cada línea con 3 campos, símbolo no vacío, color con la forma
    `#RRGGBB`, sin símbolos repetidos, y que exista el símbolo `.` (fondo).
  - Mapa: que tenga filas, que ninguna fila esté vacía, que todas tengan la
    misma longitud que la fila 0, y que cada símbolo esté en la paleta.

  Reporta **todos** los problemas (no se detiene en el primero), indicando
  línea física, fila y columna (desde 0). Si hay alguno, el programa no
  calcula regiones ni SVG.
  """

  @doc """
  Valida el mapa y la paleta.

  - `filas`: lista de `{numero_linea, [simbolos]}` leída de `mapa.txt`.
  - `colores`: lista de `{numero_linea, {:ok, %Color{}} | {:error, motivo}}`.

  Retorna la lista de problemas (vacía si todo está bien).
  """
  def validar(filas, colores) do
    {tabla, problemas_paleta} = validar_paleta(colores, %{}, [])
    problemas_fondo = validar_fondo(tabla)
    problemas_mapa = validar_mapa(filas, tabla)
    problemas_paleta ++ problemas_fondo ++ problemas_mapa
  end

  # ---------------------------------------------------------------------------
  # Paleta
  # ---------------------------------------------------------------------------

  # Recorre las líneas de la paleta construyendo la tabla simbolo => color.
  defp validar_paleta([], tabla, problemas), do: {tabla, Lista.invertir(problemas)}

  defp validar_paleta([{linea, {:error, motivo}} | resto], tabla, problemas) do
    validar_paleta(resto, tabla, ["paleta.csv línea #{linea}: #{motivo}" | problemas])
  end

  defp validar_paleta([{linea, {:ok, color}} | resto], tabla, problemas) do
    problemas =
      case Map.fetch(tabla, color.simbolo) do
        {:ok, anterior} ->
          [
```

```
            "paleta.csv línea #{linea}: el símbolo '#{color.simbolo}' está repetido " <>
              "(ya se definió en la línea #{anterior.linea})"
            | problemas
          ]

        :error ->
          problemas
      end

    problemas =
      if hex_valido?(color.hex) do
        problemas
      else
        [
          "paleta.csv línea #{linea}: el color '#{color.hex}' del símbolo '#{color.simbolo}' " <>
            "no tiene la forma #RRGGBB"
          | problemas
        ]
      end

    validar_paleta(resto, Map.put_new(tabla, color.simbolo, color), problemas)
  end

  defp validar_fondo(tabla) do
    if Map.has_key?(tabla, ".") do
      []
    else
      ["paleta.csv: falta declarar el símbolo '.' (fondo)"]
    end
  end

  @doc """
  Verifica recursivamente que un texto tenga la forma `#RRGGBB`.

  ## Ejemplos

      Mapa.Validacion.hex_valido?("#2563EB") #=> true
      Mapa.Validacion.hex_valido?("#25G3EB") #=> false
      Mapa.Validacion.hex_valido?("2563EB")  #=> false
  """
  def hex_valido?("#" <> digitos) when byte_size(digitos) == 6 do
    digitos_hex?(String.to_charlist(digitos))
  end

  def hex_valido?(_texto), do: false

  defp digitos_hex?([]), do: true

  defp digitos_hex?([caracter | resto])
       when caracter in ?0..?9 or caracter in ?a..?f or caracter in ?A..?F do
    digitos_hex?(resto)
  end

  defp digitos_hex?(_caracteres), do: false

  # ---------------------------------------------------------------------------
  # Mapa
  # ---------------------------------------------------------------------------

  defp validar_mapa([], _tabla) do
    ["mapa.txt: el mapa no tiene filas (archivo vacío o solo comentarios)"]
  end

  # El ancho esperado es el de la fila 0.
  defp validar_mapa([{_linea, primera} | _resto] = filas, tabla) do
    validar_filas(filas, Lista.contar(primera), 0, tabla, [])
  end

  defp validar_filas([], _ancho, _fila, _tabla, problemas), do: Lista.invertir(problemas)

  defp validar_filas([{linea, simbolos} | resto], ancho, fila, tabla, problemas) do
    columnas = Lista.contar(simbolos)
    ubicacion = "mapa.txt línea #{linea} (fila #{fila})"
```

```elixir

    problemas =
      cond do
        columnas == 0 ->
          ["#{ubicacion}: la fila está vacía" | problemas]

        columnas != ancho ->
          [
            "#{ubicacion}: tiene #{columnas} columnas y se esperaban #{ancho} (como la fila 0)"
            | problemas
          ]

        true ->
          problemas
      end

    problemas = validar_simbolos(simbolos, ubicacion, 0, tabla, problemas)
    validar_filas(resto, ancho, fila + 1, tabla, problemas)
  end

  defp validar_simbolos([], _ubicacion, _columna, _tabla, problemas), do: problemas

  # El fondo "." no se reporta casilla por casilla: si falta, validar_fondo/1 ya
  # lo informó una vez.
  defp validar_simbolos([simbolo | resto], ubicacion, columna, tabla, problemas) do
    problemas =
      if simbolo == "." or Map.has_key?(tabla, simbolo) do
        problemas
      else
        [
          "#{ubicacion}, columna #{columna}: el símbolo '#{simbolo}' no está definido en paleta.csv"
          | problemas
        ]
      end

    validar_simbolos(resto, ubicacion, columna + 1, tabla, problemas)
  end
end
```

lib/problema2/regiones.ex — 95 líneas

```elixir
defmodule Mapa.Regiones do
  @moduledoc """
  **Identificación recursiva de regiones** (problema 2).

  Usa las dos funciones recursivas que pide el enunciado:

  - `recorrer_casillas/5`: recorre todas las casillas en orden (fila por fila,
    columna por columna) y, cuando encuentra una casilla que no es fondo y aún
    no tiene región, abre una región nueva con el siguiente ID.
  - `explorar/5`: desde una casilla, visita recursivamente los 4 vecinos con
    el mismo símbolo (relleno por inundación, *flood fill*).

  El conjunto `asignadas` se pasa de llamada en llamada: así ninguna casilla
  se recorre dos veces.
  """

  @doc """
  Convierte las filas leídas (`[{linea, [simbolos]}]`) en la estructura del
  mapa:

      %{celdas: %{{fila, columna} => simbolo}, filas: n, columnas: m}
  """
  def construir_mapa([{_linea, primera} | _] = filas) do
    celdas =
      for {{_linea, simbolos}, fila} <- Enum.with_index(filas),
          {simbolo, columna} <- Enum.with_index(simbolos),
          into: %{},
          do: {{fila, columna}, simbolo}

    %{celdas: celdas, filas: length(filas), columnas: length(primera)}
  end

  @doc "Todas las coordenadas en orden: fila ascendente y luego columna ascendente."
```

```elixir
  def posiciones(%{filas: filas, columnas: columnas}) do
    for fila <- 0..(filas - 1)//1, columna <- 0..(columnas - 1)//1, do: {fila, columna}
  end

  @doc """
  Encuentra todas las regiones del mapa, con ID desde 1 en orden de aparición.

  ## Ejemplo (mapa del enunciado)

      Mapa.Regiones.encontrar(mapa) |> Enum.map(&Region.a_csv/1)
      #=> ["1;A;3;8;{0,0};{1,1}", "2;B;3;8;{0,3};{1,4}",
      #    "3;C;5;12;{2,0};{4,2}", "4;B;3;8;{3,3};{4,4}"]
  """
  def encontrar(mapa) do
    mapa
    |> posiciones()
    |> recorrer_casillas(mapa.celdas, MapSet.new(), 1, [])
  end

  # Caso base: no quedan casillas por revisar.
  defp recorrer_casillas([], _celdas, _asignadas, _siguiente_id, regiones) do
    Enum.reverse(regiones)
  end

  defp recorrer_casillas([posicion | resto], celdas, asignadas, siguiente_id, regiones) do
    simbolo = Map.fetch!(celdas, posicion)

    if simbolo == "." or MapSet.member?(asignadas, posicion) do
      # Fondo o casilla ya asignada: se salta.
      recorrer_casillas(resto, celdas, asignadas, siguiente_id, regiones)
    else
      # Casilla nueva: se explora toda su región.
      {casillas_region, asignadas} =
        explorar(posicion, simbolo, celdas, MapSet.put(asignadas, posicion), [])

      region = Region.nueva(siguiente_id, simbolo, casillas_region, celdas)
      recorrer_casillas(resto, celdas, asignadas, siguiente_id + 1, [region | regiones])
    end
  end

  # Visita la casilla `posicion` (ya marcada como asignada) y luego sus vecinos.
  # Retorna {casillas de la región, asignadas actualizadas}.
  defp explorar(posicion, simbolo, celdas, asignadas, region) do
    posicion
    |> Region.vecinos()
    |> explorar_vecinos(simbolo, celdas, asignadas, [posicion | region])
  end

  defp explorar_vecinos([], _simbolo, _celdas, asignadas, region), do: {region, asignadas}

  defp explorar_vecinos([vecino | resto], simbolo, celdas, asignadas, region) do
    # Map.get devuelve nil fuera del mapa, así que no hace falta revisar bordes.
    if Map.get(celdas, vecino) == simbolo and not MapSet.member?(asignadas, vecino) do
      {region, asignadas} =
        explorar(vecino, simbolo, celdas, MapSet.put(asignadas, vecino), region)

      explorar_vecinos(resto, simbolo, celdas, asignadas, region)
    else
      explorar_vecinos(resto, simbolo, celdas, asignadas, region)
    end
  end
end
```

lib/problema2/svg.ex — 95 líneas

```elixir
defmodule Mapa.Svg do
  @moduledoc """
  **Exportación SVG** del mapa de regiones (problema 2).

  El documento se genera **por fragmentos** con `Stream.concat/1`:

  1. encabezado (`<?xml ...>`, `<svg ...>` y el rectángulo de fondo),
  2. un `<rect>` de 40 × 40 px por casilla, con el color de la paleta y borde,
  3. un `<text>` con el ID de cada región, en su primera casilla,
```

```
  4. el cierre `</svg>`.

  `Salida.escribir/2` consume ese Stream y escribe cada fragmento en cuanto se
  produce; nunca existe el documento completo en memoria.

  Todo texto que viene de los archivos (nombres de la paleta, símbolos) pasa
  por `escapar_xml/1` antes de entrar al XML.
  """

  @tamano 40

  @doc "Genera el archivo SVG en `ruta`."
  def generar(ruta, mapa, regiones, paleta) do
    ancho = @tamano * mapa.columnas
    alto = @tamano * mapa.filas
    region_de = for region <- regiones, celda <- region.celdas, into: %{}, do: {celda, region}

    # Nada se calcula aquí todavía: el Stream solo describe los fragmentos.
    fragmentos =
      Stream.concat([
        encabezado(ancho, alto, paleta["."].hex),
        Stream.map(Mapa.Regiones.posiciones(mapa), &casilla(&1, mapa.celdas, paleta, region_de)),
        Stream.map(regiones, &etiqueta/1),
        ["</svg>"]
      ])

    # Al escribir se consume el Stream: cada fragmento se genera y se escribe enseguida.
    Salida.escribir(ruta, fragmentos)
  end

  defp encabezado(ancho, alto, color_fondo) do
    [
      ~s(<?xml version="1.0" encoding="UTF-8"?>),
      ~s(<svg xmlns="http://www.w3.org/2000/svg" width="#{ancho}" height="#{alto}" ) <>
        ~s(viewBox="0 0 #{ancho} #{alto}">),
      ~s(<rect x="0" y="0" width="#{ancho}" height="#{alto}" fill="#{color_fondo}"/>)
    ]
  end

  # Un rectángulo por casilla. El <title> aparece al pasar el mouse en el navegador.
  defp casilla({fila, columna} = posicion, celdas, paleta, region_de) do
    simbolo = celdas[posicion]
    color = paleta[simbolo]

    descripcion =
      case region_de[posicion] do
        nil ->
          "Fila #{fila}, columna #{columna}: #{color.nombre} (#{simbolo})"

        region ->
          "Fila #{fila}, columna #{columna}: #{color.nombre} (#{simbolo}), región #{region.id}"
      end

    ~s(<rect x="#{columna * @tamano}" y="#{fila * @tamano}" width="#{@tamano}" ) <>
      ~s(height="#{@tamano}" fill="#{color.hex}" stroke="#334155" stroke-width="1">) <>
      "<title>#{escapar_xml(descripcion)}</title></rect>"
  end

  # El ID se escribe en la primera casilla de la región (la primera en orden de filas).
  defp etiqueta(%Region{id: id, celdas: [{fila, columna} | _]}) do
    x = columna * @tamano + div(@tamano, 2)
    y = fila * @tamano + div(@tamano, 2)

    ~s(<text x="#{x}" y="#{y}" text-anchor="middle" dominant-baseline="central" ) <>
      ~s(font-family="Arial, sans-serif" font-size="18" font-weight="bold" ) <>
      ~s(fill="#0F172A" stroke="#FFFFFF" stroke-width="3" paint-order="stroke">) <>
      "#{escapar_xml(Integer.to_string(id))}</text>"
  end

  @doc """
  Escapa los caracteres especiales de XML. `&` va primero para no escapar dos
  veces los `&` que se agregan.
```

```
  ## Ejemplo

      Mapa.Svg.escapar_xml("agua & <río>") #=> "agua &amp; &lt;río&gt;"
  """
  def escapar_xml(texto) do
    texto
    |> String.replace("&", "&amp;")
    |> String.replace("<", "&lt;")
    |> String.replace(">", "&gt;")
    |> String.replace("\"", "&quot;")
    |> String.replace("'", "&apos;")
  end
end
```

lib/problema2/problema2.ex — 156 líneas

```
defmodule Problema2 do
  @moduledoc """
  **Problema 2: regiones de un mapa de colores y exportación SVG.**
  Programa principal.

  Flujo completo (descomposición del problema):

  1. Verificar que existan `mapa.txt` y `paleta.csv`.
  2. Leer ambos archivos de forma perezosa (Stream) y, ya consumidos,
     guardarlos en listas.
  3. Validar recursivamente (`Mapa.Validacion`) → `validacion_mapa.txt`.
     Si hay problemas, se detiene sin calcular regiones ni SVG.
  4. Construir el mapa y encontrar las regiones (`Mapa.Regiones`).
  5. Escribir `regiones.csv`, `mapa_regiones.svg` y `resumen_regiones.txt`.
  6. Mostrar el resumen en pantalla.

  ## Uso

      mix run problema2.exs datos/problema2/ejemplo
  """

  alias Mapa.{Regiones, Svg, Validacion}

  @entrada_por_defecto "datos/problema2/ejemplo"
  @salidas_calculadas ["regiones.csv", "mapa_regiones.svg", "resumen_regiones.txt"]

  @doc """
  Ejecuta el problema 2 con los archivos de `dir_entrada` y deja los resultados
  en `dir_salida` (por defecto `salidas/problema2/<nombre del caso>`).

  Retorna `{:ok, regiones}`, `{:error, problemas}` o
  `{:error, {:faltan_archivos, rutas}}`.
  """
  def main(dir_entrada \\ @entrada_por_defecto, dir_salida \\ nil) do
    dir_salida = dir_salida || Path.join("salidas/problema2", Path.basename(dir_entrada))
    ruta_mapa = Path.join(dir_entrada, "mapa.txt")
    ruta_paleta = Path.join(dir_entrada, "paleta.csv")

    Util2.mostrar("\n=== Problema 2: regiones del mapa (#{dir_entrada}) ===", :mensaje)

    case Entrada.verificar_archivos([ruta_mapa, ruta_paleta]) do
      :ok ->
        ejecutar(ruta_mapa, ruta_paleta, dir_salida)

      {:error, faltantes} ->
        Enum.each(faltantes, &Util2.mostrar("Falta el archivo requerido: #{&1}", :error))
        {:error, {:faltan_archivos, faltantes}}
    end
  end

  defp ejecutar(ruta_mapa, ruta_paleta, dir_salida) do
    filas = leer_mapa(ruta_mapa)
    colores = leer_paleta(ruta_paleta)

    problemas = Validacion.validar(filas, colores)
    Salida.escribir(Path.join(dir_salida, "validacion_mapa.txt"), lineas_validacion(problemas))

    case problemas do
```

```
      [] ->
        calcular(filas, colores, dir_salida)

      _ ->
        # Se borran resultados viejos para que no se confundan con esta ejecución.
        Salida.eliminar(Enum.map(@salidas_calculadas, &Path.join(dir_salida, &1)))

        Util2.mostrar(
          "#{cantidad_problemas(problemas)} (ver validacion_mapa.txt); " <>
            "no se calculan regiones ni SVG:",
          :error
        )

        Enum.each(problemas, &Util2.mostrar("  - #{&1}", :error))
        {:error, problemas}
    end
  end

  # Lectura perezosa: cada línea útil se separa en símbolos; luego se consume.
  defp leer_mapa(ruta) do
    ruta
    |> Entrada.lineas_utiles()
    |> Stream.map(fn {numero, texto} -> {numero, String.split(texto)} end)
    |> Enum.to_list()
  end

  defp leer_paleta(ruta) do
    ruta
    |> Entrada.lineas_utiles()
    |> Stream.map(fn {numero, _texto} = linea -> {numero, Color.parse(linea)} end)
    |> Enum.to_list()
  end

  defp calcular(filas, colores, dir_salida) do
    paleta = Color.tabla(colores)
    mapa = Regiones.construir_mapa(filas)
    regiones = Regiones.encontrar(mapa)

    Salida.escribir(
      Path.join(dir_salida, "regiones.csv"),
      Stream.concat([
        ["id;simbolo;area;perimetro;desde;hasta"],
        Stream.map(regiones, &Region.a_csv/1)
      ])
    )

    Svg.generar(Path.join(dir_salida, "mapa_regiones.svg"), mapa, regiones, paleta)

    resumen = lineas_resumen(regiones, paleta)
    Salida.escribir(Path.join(dir_salida, "resumen_regiones.txt"), resumen)
    Enum.each(resumen, &Util2.mostrar(&1, :mensaje))
    Util2.mostrar("Archivos generados en #{dir_salida}", :mensaje)
    {:ok, regiones}
  end

  # ---------------------------------------------------------------------------
  # Formato de reportes
  # ---------------------------------------------------------------------------

  defp lineas_validacion([]) do
    ["Validación correcta: mapa.txt y paleta.csv no tienen problemas."]
  end

  defp lineas_validacion(problemas) do
    ["#{cantidad_problemas(problemas)}:"] ++
      Util2.convertir_coleccion_mensaje(problemas, &" - #{&1}")
  end

  defp cantidad_problemas([_unico]), do: "Se encontró 1 problema"
  defp cantidad_problemas(problemas), do: "Se encontraron #{length(problemas)} problemas"

  defp lineas_resumen(regiones, paleta) do
    [
```

```
      "Número de regiones: #{length(regiones)}",
      maximos("Mayor área", {"casilla", "casillas"}, regiones, & &1.area, paleta),
      maximos("Mayor perímetro", {"lado", "lados"}, regiones, & &1.perimetro, paleta)
    ]
  end

  defp maximos(titulo, _unidades, [], _criterio, _paleta) do
    "#{titulo}: no hay regiones (el mapa solo tiene fondo)"
  end

  # Busca el valor máximo y conserva todas las regiones empatadas.
  defp maximos(titulo, {singular, plural}, regiones, criterio, paleta) do
    maximo = regiones |> Enum.map(criterio) |> Enum.max()
    unidad = if maximo == 1, do: singular, else: plural

    empatadas =
      regiones
      |> Util2.aplicar_filtro(&(criterio.(&1) == maximo))
      |> Util2.ordenar(:asc, & &1.id)
      |> Enum.map(&"región #{&1.id} (#{&1.simbolo}, #{paleta[&1.simbolo].nombre})")
      |> Enum.join(", ")

    "#{titulo}: #{maximo} #{unidad} → #{empatadas}"
  end
end
```

Pruebas (ExUnit)

test/test_helper.exs — 31 líneas

```
ExUnit.start()

defmodule Apoyo do
  @moduledoc "Funciones de apoyo para las pruebas."

  @doc "Carpeta temporal única para que cada prueba escriba sus salidas."
  def carpeta_temporal(nombre) do
    ruta =
      Path.join(System.tmp_dir!(), "parcial2_#{nombre}_#{System.unique_integer([:positive])}")

    File.rm_rf!(ruta)
    ruta
  end

  @doc "Escribe un archivo de entrada de prueba (crea la carpeta)."
  def escribir(carpeta, nombre, contenido) do
    File.mkdir_p!(carpeta)
    File.write!(Path.join(carpeta, nombre), contenido)
  end

  @doc """
  Lee un archivo completo como texto. Incluso en las pruebas se lee con
  `File.stream!/1` (requisito general 1): en todo el proyecto no se usa `File.read!/1`.
  """
  def leer(ruta), do: ruta |> File.stream!() |> Enum.join()

  @doc "Lee un archivo de salida como lista de líneas."
  def lineas(carpeta, nombre) do
    carpeta |> Path.join(nombre) |> leer() |> String.split("\n", trim: true)
  end
end
```

test/problema1_test.exs — 181 líneas

```
defmodule Problema1Test do
  use ExUnit.Case
  import ExUnit.CaptureIO

  alias Rutas.{Alcance, Busqueda, Depuracion}

  # Red del enunciado ya depurada (sin A,A ni el segundo A,B).
  defp red_ejemplo do
    [
```

```elixir
      {2, "A,B,8,5"},
      {3, "B,D,7,4"},
      {4, "A,C,6,2"},
      {5, "C,D,12,1"},
      {6, "B,C,3,2"},
      {7, "C,B,4,1"}
    ]
    |> Enum.map(fn linea -> linea |> Tramo.parse() |> elem(1) end)
  end

  describe "Tramo.parse/1" do
    test "convierte una línea válida" do
      assert {:ok, %Tramo{origen: "A", destino: "B", minutos: 8, costo: 5}} =
               Tramo.parse("A,B,8,5")
    end

    test "conserva la línea física y tolera espacios" do
      assert {:ok, %Tramo{origen: "A", destino: "B", linea: 7}} =
               Tramo.parse({7, " A , B , 8 , 5 "})
    end

    test "rechaza cantidad de campos incorrecta" do
      assert {:error, motivo} = Tramo.parse("C,D,5")
      assert motivo =~ "4 campos"
    end

    test "rechaza valores no numéricos, cero, negativos y centros vacíos" do
      assert {:error, _} = Tramo.parse("B,C,ocho,4")
      assert {:error, _} = Tramo.parse("B,C,8.5,4")
      assert {:error, _} = Tramo.parse("D,E,0,3")
      assert {:error, _} = Tramo.parse("E,F,4,-2")
      assert {:error, _} = Tramo.parse(",F,3,3")
    end
  end

  describe "depuración recursiva" do
    test "descarta bucles y repetidos, conserva la primera aparición y los sentidos contrarios" do
      tramos =
        [{2, "A,B,8,5"}, {8, "A,A,2,1"}, {9, "A,B,9,7"}, {10, "B,A,1,1"}]
        |> Enum.map(&(&1 |> Tramo.parse() |> elem(1)))

      {conservados, descartes} = Depuracion.depurar(tramos)

      assert Enum.map(conservados, & &1.linea) == [2, 10]
      assert [{%Tramo{linea: 8}, bucle}, {%Tramo{linea: 9}, repetido}] = descartes
      assert bucle =~ "bucle"
      assert repetido =~ "línea 2"
    end

    test "el descarte conserva la línea física y el contenido original, con sus espacios" do
      tramos =
        Enum.map([{2, "A,B,8,5"}, {6, " A , B , 9 , 7 "}], &(&1 |> Tramo.parse() |> elem(1)))

      assert {_, [{%Tramo{linea: 6, contenido: " A , B , 9 , 7 "}, _motivo}]} =
               Depuracion.depurar(tramos)
    end

    test "lista vacía" do
      assert Depuracion.depurar([]) == {[], []}
    end

    test "requisito 7: el módulo de validación no usa Enum ni Stream" do
      codigo = Apoyo.leer("lib/problema1/depuracion.ex")
      refute codigo =~ "Enum."
      refute codigo =~ "Stream."
    end
  end

  describe "búsqueda de itinerarios" do
    test "encuentra los 4 itinerarios del enunciado, en el mismo orden" do
      grafo = Busqueda.construir_grafo(red_ejemplo())
      itinerarios = Busqueda.itinerarios(grafo, "A", "D")

```

```elixir
    assert Enum.map(itinerarios, &to_string/1) == [
             "A → B → D | tramos: 2 | 15 min | costo: 9",
             "A → B → C → D | tramos: 3 | 23 min | costo: 8",
             "A → C → D | tramos: 2 | 18 min | costo: 3",
             "A → C → B → D | tramos: 3 | 17 min | costo: 7"
           ]

    assert Busqueda.optimos(itinerarios) == %{tramos: [1, 3], minutos: [1], costo: [3]}
  end

  test "origen igual a destino da un itinerario de 0 tramos" do
    grafo = Busqueda.construir_grafo(red_ejemplo())

    assert [%Itinerario{centros: ["A"], minutos: 0, costo: 0}] =
             Busqueda.itinerarios(grafo, "A", "A")
  end

  test "conserva empates en los tres criterios" do
    tramos =
      Enum.map(["A,B,1,1", "B,D,1,1", "A,C,1,1", "C,D,1,1"], &(&1 |> Tramo.parse() |> elem(1)))

    itinerarios = tramos |> Busqueda.construir_grafo() |> Busqueda.itinerarios("A", "D")
    assert Busqueda.optimos(itinerarios) == %{tramos: [1, 2], minutos: [1, 2], costo: [1, 2]}
  end
end

describe "Stream infinito de alcance" do
  test "se estabiliza en el paso 2 para A" do
    grafo = Busqueda.construir_grafo(red_ejemplo())
    {estados, paso} = Alcance.hasta_estabilizar(grafo, "A")

    assert paso == 2
    assert Enum.map(estados, &Alcance.formatear/1) == ["{A}", "{A, B, C}", "{A, B, C, D}"]
  end

  test "A no es alcanzable desde B" do
    grafo = Busqueda.construir_grafo(red_ejemplo())
    {estados, _paso} = Alcance.hasta_estabilizar(grafo, "B")
    refute MapSet.member?(List.last(estados), "A")
  end

  test "el Stream es infinito: se pueden pedir tantos pasos como se quiera" do
    grafo = Busqueda.construir_grafo(red_ejemplo())
    assert grafo |> Alcance.secuencia("A") |> Enum.take(50) |> length() == 50
  end
end

describe "programa completo" do
  test "caso del enunciado" do
    salida = Apoyo.carpeta_temporal("p1_ejemplo")

    capture_io(fn ->
      assert {:ok, [{"A,D", 4}, {"B,A", 0}]} = Problema1.main("datos/problema1/ejemplo", salida)
    end)

    itinerarios = Apoyo.lineas(salida, "itinerarios.txt")
    assert "Itinerario 1: A → B → D | tramos: 2 | 15 min | costo: 9" in itinerarios
    assert "Menos tramos: 1, 3" in itinerarios
    assert "Menor tiempo: 1" in itinerarios
    assert "Menor costo: 3" in itinerarios

    depuracion = Apoyo.lineas(salida, "depuracion_tramos.txt")
    assert Enum.any?(depuracion, &String.starts_with?(&1, "Línea 8: A,A,2,1"))
    assert Enum.any?(depuracion, &String.starts_with?(&1, "Línea 9: A,B,9,7"))
  end

  test "caso inválido: reporta errores con su línea física" do
    salida = Apoyo.carpeta_temporal("p1_invalido")

    capture_io(:stderr, fn ->
      capture_io(fn -> Problema1.main("datos/problema1/invalido", salida) end)
    end)
```

```
      errores = Apoyo.lineas(salida, "errores_tramos.txt")
      assert Enum.any?(errores, &String.starts_with?(&1, "Línea 3: B,C,ocho,4"))
      assert Enum.any?(errores, &String.starts_with?(&1, "Línea 7: ,F,3,3"))

      itinerarios = Apoyo.leer(Path.join(salida, "itinerarios.txt"))
      assert itinerarios =~ "el centro Z no existe"
      assert itinerarios =~ "Consulta inválida (línea 5)"
    end

    test "archivo faltante: informa cuál falta y no lanza excepción" do
      salida = Apoyo.carpeta_temporal("p1_faltante")

      error =
        capture_io(:stderr, fn ->
          capture_io(fn ->
            assert {:error, {:faltan_archivos, [faltante]}} =
                     Problema1.main("datos/problema1/faltante", salida)

            assert faltante =~ "consultas.txt"
          end)
        end)

      assert error =~ "Falta el archivo requerido"
    end
  end
end
```

test/problema2_test.exs — 163 líneas

```
defmodule Problema2Test do
  use ExUnit.Case
  import ExUnit.CaptureIO

  alias Mapa.{Regiones, Svg, Validacion}

  defp filas(texto) do
    texto
    |> String.split("\n", trim: true)
    |> Enum.with_index(1)
    |> Enum.map(fn {linea, n} -> {n, String.split(linea)} end)
  end

  defp paleta(lineas) do
    lineas |> Enum.with_index(1) |> Enum.map(fn {t, n} -> {n, Color.parse({n, t})} end)
  end

  @paleta_ok ["A,agua,#2563EB", "B,bosque,#16A34A", "C,cultivo,#F59E0B", ".,fondo,#F1F5F9"]

  describe "validación recursiva" do
    test "datos correctos no reportan problemas" do
      assert Validacion.validar(filas("A A\n. B"), paleta(@paleta_ok)) == []
    end

    test "reporta todos los problemas de mapa y paleta" do
      problemas =
        Validacion.validar(
          filas("A A . B\nA . X B B\n. . C ."),
          paleta([
            "A,agua,#2563EB",
            "B,bosque,#16A34",
            "C,cultivo,#F59E0B",
            "A,arena,#FDE68A",
            "D,desierto"
          ])
        )

      texto = Enum.join(problemas, "\n")
      assert texto =~ "no tiene la forma #RRGGBB"
      assert texto =~ "'A' está repetido"
      assert texto =~ "3 campos"
      assert texto =~ "falta declarar el símbolo '.'"
      assert texto =~ "tiene 5 columnas y se esperaban 4"
      assert texto =~ "columna 2: el símbolo 'X'"
    end
```

```elixir

  test "mapa sin filas" do
    assert ["mapa.txt: el mapa no tiene filas" <> _] =
             Validacion.validar([], paleta(@paleta_ok))
  end

  test "formato hexadecimal" do
    assert Validacion.hex_valido?("#2563eb")
    refute Validacion.hex_valido?("#2563E")
    refute Validacion.hex_valido?("#2563EG")
    refute Validacion.hex_valido?("2563EBA")
  end

  test "requisito 7: el módulo de validación no usa Enum ni Stream" do
    codigo = Apoyo.leer("lib/problema2/validacion.ex") <> Apoyo.leer("lib/comun/lista.ex")
    refute codigo =~ "Enum."
    refute codigo =~ "Stream."
  end
end

describe "regiones" do
  test "mapa del enunciado" do
    mapa =
      Regiones.construir_mapa(filas("A A . B B\nA . . B .\n. . C . .\nC C C . B\nC . . B B"))

    assert mapa |> Regiones.encontrar() |> Enum.map(&Region.a_csv/1) == [
             "1;A;3;8;{0,0};{1,1}",
             "2;B;3;8;{0,3};{1,4}",
             "3;C;5;12;{2,0};{4,2}",
             "4;B;3;8;{3,3};{4,4}"
           ]
  end

  test "las diagonales no unen regiones" do
    mapa = Regiones.construir_mapa(filas("A .\n. A"))
    assert length(Regiones.encontrar(mapa)) == 2
  end

  test "solo fondo: ninguna región" do
    assert Regiones.encontrar(Regiones.construir_mapa(filas(". .\n. ."))) == []
  end

  test "una región grande no repite casillas" do
    mapa =
      Regiones.construir_mapa(filas(String.duplicate(String.duplicate("A ", 30) <> "\n", 30)))

    assert [%Region{area: 900, perimetro: 120}] = Regiones.encontrar(mapa)
  end
end

test "escape XML" do
  assert Svg.escapar_xml(~s(agua & "río" <dulce>)) == "agua &amp;amp; &amp;quot;río&amp;quot; &amp;lt;dulce&amp;gt;"
end

describe "programa completo" do
  test "caso del enunciado: CSV exacto y SVG bien formado" do
    salida = Apoyo.carpeta_temporal("p2_ejemplo")
    capture_io(fn -> assert {:ok, _} = Problema2.main("datos/problema2/ejemplo", salida) end)

    assert Apoyo.lineas(salida, "regiones.csv") == [
             "id;simbolo;area;perimetro;desde;hasta",
             "1;A;3;8;{0,0};{1,1}",
             "2;B;3;8;{0,3};{1,4}",
             "3;C;5;12;{2,0};{4,2}",
             "4;B;3;8;{3,3};{4,4}"
           ]

    svg = Apoyo.leer(Path.join(salida, "mapa_regiones.svg"))
    assert svg =~ ~s(width="200" height="200")
    # 25 casillas + 1 rectángulo de fondo
    assert length(Regex.scan(~r/<rect /, svg)) == 26
    assert length(Regex.scan(~r/<text /, svg)) == 4
    # Se puede leer como XML válido
```

```
      assert {_, _} = svg |> :binary.bin_to_list() |> :xmerl_scan.string(quiet: true)
    end

    test "caso inválido: no genera regiones ni SVG" do
      salida = Apoyo.carpeta_temporal("p2_invalido")

      capture_io(:stderr, fn ->
        capture_io(fn ->
          assert {:error, [_ | _]} = Problema2.main("datos/problema2/invalido", salida)
        end)
      end)

      assert File.exists?(Path.join(salida, "validacion_mapa.txt"))
      refute File.exists?(Path.join(salida, "regiones.csv"))
      refute File.exists?(Path.join(salida, "mapa_regiones.svg"))
    end

    test "empates y caracteres especiales en la paleta" do
      salida = Apoyo.carpeta_temporal("p2_empates")

      capture_io(fn ->
        assert {:ok, regiones} = Problema2.main("datos/problema2/empates", salida)
        assert length(regiones) == 5
      end)

      [_numero, area, _perimetro] = Apoyo.lineas(salida, "resumen_regiones.txt")
      assert area =~ "región 1" and area =~ "región 5"
      svg = Apoyo.leer(Path.join(salida, "mapa_regiones.svg"))
      assert svg =~ "agua &amp;amp; &amp;quot;rio&amp;quot; &amp;lt;dulce&amp;gt;"
      assert {_, _} = svg |> :binary.bin_to_list() |> :xmerl_scan.string(quiet: true)
    end

    test "archivo faltante: informa cuál falta" do
      error =
        capture_io(:stderr, fn ->
          capture_io(fn ->
            assert {:error, {:faltan_archivos, _}} =
                     Problema2.main("datos/problema2/faltante", Apoyo.carpeta_temporal("p2_f"))
          end)
        end)

      assert error =~ "paleta.csv"
    end
  end
end
```